Deployment Guide

Revision 2.0 | Updated 25 October 2026 | 1 / 6

System overview and release preparation

Target system

Production: https://www.aucklandknights.co.nz. GitHub: volgasystems/auckland-knights-chess-club. Vercel team: volgasystems-projects; project: auckland-knights-chess-club. The application uses Next.js/TypeScript/Tailwind, Supabase Postgres/Auth, Stripe Checkout and configured email services. Work only on AKCC; HPCC and the native mobile app have separate project boundaries.

Existing production first

Inspect the current branch, database schema, deployment configuration and migration history before changing anything. Back up production data using the available Supabase facilities. Never run reset_database.sql or recreate application tables to perform an upgrade. Do not overwrite existing users, payments, membership IDs or policy/consent snapshots.

Install and validate

Use the repository lockfile with npm ci. Run npx tsc --noEmit and node --test tests/*.test.cjs. Run npm run build with the required environment configured. Local compilation alone is not a production deployment; inspect the Vercel build result and the live route behaviour.

Runtime and secrets

Check the current Vercel-supported patched Node runtime and redeploy after relevant runtime/configuration changes. The earlier runtime security check was not completed; this guide does not certify the runtime patch level. Keep environment secrets in the appropriate production/preview configuration and restrict access. Do not publish keys, real bank details or member records in GitHub or these public PDFs.

Guide map

Pages 2-6 cover environment variables, database upgrades, Stripe/bank transfer, email/reminders, deployment checks and recovery. Document revision 2.0 dated 25 October 2026 replaces the May guide; it is an operating reference, not evidence that optional services are activated.

Deployment Guide

Environment, authentication and access

Required backend variables

Set NEXT_PUBLIC_SUPABASE_URL and NEXT_PUBLIC_SUPABASE_ANON_KEY to the intended Supabase project. Set SUPABASE_SERVICE_ROLE_KEY only on the server. NEXT_PUBLIC_SITE_URL must be the preferred production HTTPS origin: https://www.aucklandknights.co.nz. Use separate test/preview configuration; never expose a service key in client bundles.

Stripe variables

Production checkout requires a live STRIPE_SECRET_KEY, a configured STRIPE_WEBHOOK_SECRET beginning whsec_, and a valid production site URL. Do not use live keys for automated tests. The application refuses unsafe/missing production checkout configuration. Bank transfer follows its own database/account readiness checks rather than requiring Stripe.

Email and address variables

Configure email in Club Settings and/or the supported server environment. Current code supports SMTP/Brevo and Resend, including SMTP_HOST, SMTP_PORT, SMTP_USER, SMTP_PASS or RESEND_API_KEY, plus CLUB_FROM_EMAIL/CLUB_FROM_NAME and optional reply-to settings. Inspect lib/email.ts for precedence. NZ Post address lookup uses server-side NZPOST_CLIENT_ID and NZPOST_CLIENT_SECRET. Do not store secret values in downloadable documentation.

Admin authentication

Create/reuse the Supabase Auth account and its active profile with the minimum appropriate role. Admin routes perform server-side checks, not just sidebar hiding. Inactivity expiry is 10 minutes with activity renewal and a signed server cookie. Profile settings edit name/phone; password change requires the current password and a subsequent fresh login. Check redirect URLs use the production origin rather than localhost.

Data boundaries

Use the service client only after appropriate authorisation or narrowly validated public operations. Public entries/Excel export contain safe paid-confirmed player fields; never expose DOB, contact details, payment identifiers or consent snapshots. Documentation PDFs are public static assets even though the admin menu links to them.

Deployment Guide

Database upgrades and compatibility

Fresh installation versus upgrade

supabase/schema.sql is a reference/bootstrap for a new intended database, not a reset procedure for existing production. Existing installations should apply the needed versioned migrations after review. Preserve existing records and verify columns/functions instead of assuming a successful web deployment applied SQL.

Earlier feature migrations

Check existing requirements in migration_v4_4_membership_numbering.sql, migration_v4_5_event_gallery.sql and migration_v4_6_membership_renewals.sql. Other historical migrations may already be present. Apply only what the actual schema requires; do not blindly replay all old scripts.

Recent migrations

20261009_payment_accounts.sql introduces payment account/event linkage. 20261012_tournament_registration_gender.sql adds tournament gender support. 20261013_manual_tournament_payments.sql adds audited offline tournament approval. 20261018_tournament_consent_policy.sql stores policy/photo audit and tightens public access. 20261018_tournament_payment_reminders.sql adds reminder settings, timestamps and failure reason fields. Files are in supabase/migrations/.

Bank transfer migration

Run the complete 20261025_bank_transfer_registration.sql in the AKCC Supabase SQL Editor to activate new membership/tournament bank registrations. It adds payment method, saved bank instructions and reference fields, default account flags, uniqueness protections and audited membership approval. It includes the manual tournament function/receipt audit, so the earlier manual-payment script is not separately required for this feature. It does not mark entries paid.

Legacy columns and verification

Deployed accounts may have account_name, bank_account_name or bank_account_number alongside canonical name/account_number. The current admin API detects each alias independently and copies validated input to all detected aliases on create/update. No further SQL is needed solely for that application compatibility fix. Missing bank schema disables the registration option safely. Verify RLS, service-only RPC grants, existing data preservation and a schema reload before using new features.

Deployment Guide

Payments: Stripe, bank transfers and audits

Stripe fulfillment

Configure /api/stripe/webhook with the event subscriptions used by app/api/stripe/webhook/route.ts. Keep the signing secret matched to that endpoint. Verify completed/async success, failure/expiry and charge.refunded handling as required by the current code. Return pages and admin reconciliation also verify payment metadata, currency, amount, mode and the linked record. A redirect alone is not payment proof.

Configure bank accounts

After the bank migration, add the real account through Admin > Payment Accounts; mark Active and choose membership and/or tournament defaults. Only one active default is allowed for each type. A tournament-specific account overrides the default. Multiple unselected accounts are not guessed. Store actual bank details only in the database. Assigned accounts are protected from deletion; deactivate or reassign deliberately.

Pending state and public visibility

A bank registration uses server-calculated fees and a saved bank account snapshot plus unique M/T-prefixed reference. It returns instructions on screen and in the acknowledgement email without creating Stripe Checkout. Payment/registration stays pending until verified. A tentative tournament registration does not reserve a place. Public confirmed lists and received-payment charts exclude pending entries.

Audited confirmation

Only Admin/Super Admin may approve an offline receipt, with method, 5-500 character note and full-payment acknowledgement. Database operations record the receipt and approver atomically and handle repeated approvals without another receipt. Tournament approval checks capacity and an already confirmed player. A linked completed Stripe payment must be verified as Stripe, not overwritten as cash; an open unpaid session is expired before offline approval where applicable.

Recovery and scope

A failed membership activation can be retried from the existing bank confirmation. Never request a second payment because an email failed. For duplicates, verify the full refund and reconcile the correct registration while keeping the valid paid entry. No automatic bank-feed matching is implemented. The existing separate membership renewal checkout remains Stripe-based; new member bank registration does not change renewal fulfillment.

Deployment Guide

Email, reminder scheduler and diagnostics

Provider configuration

Choose the intended provider, verified sender and reply-to in Club Settings/server configuration. Send an authorised diagnostic test to a controlled address. Registration acknowledgement, verified-payment confirmation and reminder delivery have separate log records. Fix provider settings before retrying failed confirmation emails; never infer payment status from inbox arrival.

Activate reminders

Apply 20261018_tournament_payment_reminders.sql. Set CRON_SECRET to a securely generated server secret of at least 32 characters in the production Vercel project and redeploy. vercel.json schedules GET /api/cron/tournament-payment-reminders at 0 19 * * * (19:00 UTC daily). This is 08:00 NZDT or 07:00 NZST the following local day. Confirm the Vercel plan/job configuration actually schedules and authenticates the route.

Eligibility and timing

The tournament must be open, allow registration, enable reminders and have a valid start/deadline. The earlier of registration close and tournament start stops reminders. The first eligible reminder is after 24 hours; repeats are at least three days apart, with a possible final reminder during the last day and minimum one-day spacing. Daily execution does not guarantee an email at the exact 24-hour anniversary.

Safety and bounded processing

Each run inspects payment state before sending and suppresses confirmed, refunded, processing, duplicate confirmed players and full events. Bank-transfer entries are excluded from Stripe reminders and resume links. Claims/log identifiers prevent concurrent runs from blindly resending. Current work is bounded to 100 sends and roughly 240 seconds per run with a 300 second route limit and a candidate limit. Inspect the source if volume approaches these bounds.

Evidence and failures

Use Tournament Registrations > Last payment reminder / Payment status explanation, Email Diagnostics and Vercel cron execution logs. Sent is provider acceptance, not inbox delivery or an open/click. Inspect unsigned requests for 401; missing schema/configuration must not silently send. Manual Send due reminders sends real email after checks; use it only for an authorised run. Prior deployment did not itself confirm production CRON_SECRET or scheduler activation.

Deployment Guide

Revision 2.0 | Updated 25 October 2026 | 6 / 6

Deployment, smoke checks and recovery

Release workflow

Commit reviewed changes to the intended AKCC production branch and inspect the associated Vercel build. A failed build leaves the previous successful production version serving. After success, verify public pages and protected admin redirects, then sign in with an authorised account for operational checks. Apply SQL separately where required and record the result.

Read-only smoke checks

Confirm /, /join, /tournaments and a real event/registration/entries page load; /club-admin/accounts and /club-admin/dashboard redirect unauthenticated visitors to login. Verify each updated PDF opens and downloads. Check sitemap.xml and robots.txt, canonical origin, www/non-www HTTPS redirects, contact links and structured club identity. A successful sitemap is not a ranking guarantee.

Controlled functional checks

Use a safe test environment/account and Stripe test mode for registration/payment tests. Verify required fields, club versus general eligibility, photo Yes/No, saved refund policy, pending states, bank reference/instructions, admin-only approval, receipts, duplicates and replay behaviour. Do not approve real payments, issue refunds, delete records or email real players as a deployment smoke test.

Troubleshooting

Account NOT NULL errors: check canonical/legacy alias compatibility and the exact deployed build. Bank transfer disabled: check migration and active account/default/event assignment. Paid but absent publicly: reconcile linked Stripe/full-refund state and require paid plus confirmed. Session/page failure: reload and sign in. Email failed: review provider logs independently. Graphs empty: check period, tournament scope, permissions and query alerts before assuming no data.

Recovery and maintenance

Roll back to a known-good application deployment if needed; do not reverse additive SQL by deleting production data. Keep backups and migration notes. Update this generator and rebuild both PDFs when operating behaviour changes. Check stored secrets, runtime patch level, webhook health, reminder execution and restore readiness through the relevant provider tools. Source references: lib/roles.ts, lib/paymentAccounts.ts, lib/bankTransfer.ts, lib/tournamentPaymentReminder.ts, docs/*.md and supabase/migrations/.